SENTINEL-TS

Hybrid Time-Series Anomaly Intelligence

Forecasting, anomaly detection and decision support
for cloud infrastructure telemetry

Mini Project Report

Time Series Data Analysis (TSDA)

Department of Computer Engineering
Sardar Patel Institute of Technology

Group members

Contents

Abstract

Cloud services emit continuous telemetry, and incidents appear in that stream before customers notice them. Fixed thresholds miss those incidents for a specific and expensive reason: server load is strongly periodic, so a ceiling calibrated for the daily peak is blind to a failure at 04:00 when load legitimately collapses, while a ceiling calibrated for the overnight trough fires every weekday morning. Teams respond by widening thresholds until the noise stops, at which point the detector's real-world recall is zero.

SENTINEL-TS addresses this by modelling what each metric should be at each moment and alerting on the gap, so the decision boundary moves with time of day instead of ignoring it. Seven approaches are compared on identical chronological splits of real Amazon CloudWatch telemetry: two deliberately simple baselines, two classical time-series models, one unsupervised machine-learning detector, and two variants of a two-stage hybrid that corrects a SARIMAX forecast with a nonlinear residual learner before fusing two complementary detection channels.

The system is delivered as a working application rather than a notebook: a Python analysis package, a REST API with asynchronous job execution, and a browser console that turns raw anomaly flags into classified incidents with recommended operator actions and a forward capacity outlook. The implementation is covered by **101 automated tests** plus a headless console smoke test, several of which found genuine defects described in this report.

The headline empirical result is reported as found rather than as hoped for. Across the labelled corpus the **simple rolling z-score detector attains the highest composite operational score (0.787)**, ahead of the hybrid (0.707). The hybrid is nonetheless the quietest detector that still recovers every labelled incident window, at 1.36 false alarms per day against the baseline's 2.52 — a trade that matters more in practice than the composite ordering suggests. Section 8 examines why, and what it implies.

1. Problem Identification

1.1 The real-world problem

Site-reliability engineering teams instrument every service they own. A mid-sized platform emits thousands of metric series — CPU and memory per host, request counts per endpoint, queue depths, disk throughput, database connection pools — each sampled every few seconds to few minutes. The purpose of that instrumentation is to catch degradation before customers experience it.

In practice most of this telemetry is monitored by static thresholds, because static thresholds are trivial to configure and trivial to reason about. The failure mode that follows is well known to anyone who has carried a pager.

- Infrastructure load is strongly diurnal and weekly. A production API may run at 70% CPU during business hours and 20% overnight.
- A threshold set at 85% to avoid overnight noise cannot see a service that has silently dropped from 20% to 2% at 03:00 because an upstream dependency died — a severe incident with no threshold crossing at all.
- A threshold set tight enough to catch that would fire every single morning as load legitimately ramps.
- The usual resolution is to widen thresholds until paging stops. The alert is then still configured, still shows green on a dashboard, and detects nothing. Its effective recall is zero while appearing to provide coverage.

The problem is therefore not insufficient data or insufficient alerting. It is that the decision rule is stationary while the process it monitors is not.

1.2 Intended users

User	What they need from the system
Site-reliability / platform engineer	To know that a service is behaving abnormally for this time of day, with enough context to begin diagnosis rather than begin investigating the alert itself.
On-call engineer	A small number of pages that are worth waking up for, each carrying a classified incident shape and a concrete first action.
Capacity / infrastructure planner	A forward forecast with an uncertainty band, and an estimate of when a service ceiling is likely to be reached.
Engineering manager	Evidence that monitoring coverage is real, expressed as detection rate and alert burden rather than as a count of configured alerts.

Table 1.1 — Intended users and their requirements.

1.3 Objectives

1. Build a forecasting model of normal behaviour that accounts for daily periodicity, slow level drift and short-memory autocorrelation, so that the expected value is time-dependent rather than constant.
2. Detect anomalies as departures from that expectation, using a threshold calibrated on training data alone so the detector cannot adapt to the incidents it is meant to find.
3. Design and evaluate a hybrid model that combines a linear time-series model with a nonlinear learner, and quantify what the hybridisation actually contributes rather than assuming it helps.
4. Compare at least three approaches — in the event, seven — on identical splits, under metrics that reflect operational cost, including alert burden and detection latency, not accuracy alone.
5. Deliver a working interactive application with frontend and backend components in which a user supplies input, triggers analysis, and explores results, rather than a static notebook.
6. Convert detections into decision support: classified incident shapes, severity, deviation from forecast, recommended first actions, and a forward capacity outlook with an explicit breach estimate.

1.4 Expected outcomes

A system that detects every labelled incident window in a hold-out period it has never seen, at an alert volume an on-call rota would tolerate; a defensible, measured comparison that states plainly where the added modelling complexity pays for itself and where it does not; and an application in which each of those claims can be reproduced by a user in a few clicks.

2. Dataset Collection and Preparation

2.1 Source and provenance

The corpus is the realAWSCloudwatch family of the Numenta Anomaly Benchmark (NAB), an openly published benchmark for streaming anomaly detection. These are genuine Amazon CloudWatch measurements from production Amazon Web Services infrastructure, not simulated traces. They were selected over a synthetic generator for a specific reason: synthetic data contains exactly the structure its author put there, so a model that assumes that structure will always appear to succeed. Real telemetry contains flat idle stretches, reporting gaps, mixed noise regimes and anomalies of several different shapes, and it penalises assumptions that do not hold.

Attribute	Value
Source	Numenta Anomaly Benchmark, realAWSCloudwatch family
Repository	github.com/numenta/NAB
Nature	Production Amazon CloudWatch server telemetry
Sampling frequency	5 minutes (native)
Duration per series	≈ 14 days
Observations per series	4,032 raw (4,731 for one longer series)
Number of series used	6
Attributes per series	2 — timestamp (ISO datetime), value (float)
Ground truth	Human-curated anomaly windows, 11 across the corpus
Licence	AGPL-3.0; redistributed unmodified under data/raw/

Table 2.1 — Dataset provenance.

2.2 The six series

Series	Service (console label)	Unit	Windows
ec2_cpu_utilization_5f5533	api-gateway-prod-01	%	2
rds_cpu_utilization_cc0c53	orders-db-primary	%	2
ec2_network_in_257a54	edge-ingest-02	bytes	1
elb_request_count_8c0756	public-lb-eu-west	req/bucket	2
ec2_disk_write_bytes_1ef3de	batch-worker-07	bytes	1
grok_asg_anomaly	asg-checkout-fleet	units	3

Table 2.2 — Series in the corpus. Service names are presentational; the underlying measurements are unmodified.

2.3 Why window labels, not point labels

NAB labels anomalies as time windows rather than individual points, and this choice propagates into every evaluation decision made later in this report. It is also the operationally correct framing: an engineer cares that an incident was detected inside its window, not that one particular five-minute bucket was flagged. A detector that fires once, early, inside a six-hour incident has done its job completely — the human is now engaged.

One consequence must be stated because it affects the headline numbers. NAB windows deliberately *open before* the visibly anomalous behaviour, so that early detection is rewarded. Measured detection latency against the window start is therefore systematically pessimistic: a latency of several hours against the window boundary may be prompt against the point at which a human would have noticed anything. No correction has been applied to compensate; the figures in Section 5 are raw.

2.4 Preprocessing pipeline

Every transformation is recorded in an audit trail that the console displays, because in an operational setting it must be possible to state which values the model saw and which were reconstructed. The pipeline is implemented in sentinel/preprocessing.py.

2.4.1 Duplicate timestamps

Collapsed by taking the mean, not by dropping one of the pair. Double-reporting by a monitoring agent is common and both readings carry information about the interval.

2.4.2 Non-finite readings

Infinities and NaNs are masked before resampling so they cannot contaminate a bucket mean. They are counted in the audit trail.

2.4.3 Resampling

The native 5-minute series is resampled to 15-minute buckets by mean. This is not cosmetic. At 5 minutes the daily period is m = 288; at 15 minutes it is m = 96. Section 4 explains why that distinction determines which models are estimable at all. Resampling also reduces 4,032 observations to 1,345, which keeps a full seven-model comparison inside interactive latency (roughly 20 to 40 seconds). The frequency is user-selectable in the console between 5 minutes and 1 hour; 15 minutes is the default rather than a constraint.

2.4.4 Gap filling, by gap length

Empty buckets are not all filled the same way, because a five-minute gap and a four-hour outage are different phenomena.

- Gaps of up to 6 buckets are filled by time-aware linear interpolation.
- Longer gaps are reconstructed from the seasonal profile — the median value at the corresponding position in the daily cycle. Linear interpolation across a multi-hour outage would draw a straight line through a region where the true signal has a pronounced daily shape, and that straight line would itself read as an anomaly to any detector downstream.

2.4.5 Outliers are flagged, never removed

This is the single most consequential preprocessing decision in an anomaly-detection project, and it runs opposite to standard practice in forecasting. In this problem the extreme values are the signal. Removing or winsorising them would remove the phenomenon under study. Points beyond four robust standard deviations are therefore flagged for display and carried forward unchanged; only the statistics used to fit and threshold the models are made robust to them, through median- and MAD-based estimators rather than mean and standard deviation.

2.5 Preprocessing outcome

For the EC2 CPU series at the default configuration:

Quantity	Value
Raw observations	4,032
Observations after resampling to 15 min	1,345
Span	14.0 days
Coverage (non-empty buckets)	100.0%
Duplicate timestamps collapsed	0
Non-finite readings masked	0
Buckets interpolated	0
Buckets reconstructed seasonally	0
Extreme values flagged (|robust z| > 4)	38 (2.83%)
Seasonal period	96 buckets/day

Table 2.3 — Preprocessing audit trail, ec2_cpu_utilization_5f5533 at 15-minute buckets.

This series is complete: no gaps, no duplicates, no invalid readings. That is a useful finding in itself, because it means the model comparison in Section 5 is not confounded by imputation artefacts. The gap-filling machinery still matters — user-uploaded series routinely need it, and other corpora have real outages — but on this data it was not exercised.

3. Exploratory Data Analysis

The EDA is not decorative. Each result below is used to make a specific, named modelling decision later in the report, and the console reproduces all of them interactively for any series.

3.1 Decomposition

STL (Seasonal-Trend decomposition using Loess) is used in preference to classical additive decomposition. The reason is directly tied to Section 2.4.5: a classical decomposition centres a moving average, which a single large spike distorts badly, whereas STL fits a locally weighted regression that is robust to exactly the extremes this project is hunting. Using a method that the anomalies break would compromise every downstream estimate.

Component	Variance share	Strength measure
Trend	93.0%	0.909
Seasonal (daily)	1.7%	0.116
Remainder	9.0%	—

Table 3.1 — STL variance attribution, EC2 CPU series. Strength measures follow Hyndman and Athanasopoulos and are bounded to [0, 1].

This is an instructive result precisely because it does not match the motivating story. On this particular series the **daily cycle is weak** (seasonal strength 0.116) and the series is dominated by slow level drift (trend strength 0.909). Other series in the corpus — notably the load-balancer request count — show the strong diurnal profile described in Section 1. Reporting the weak case rather than only the convenient one matters: it means seasonal modelling is not universally the dominant consideration, and it is part of why a seasonality-blind baseline performs as well as it does in Section 5.

3.2 Stationarity

Two tests with opposite null hypotheses are run, because declaring stationarity from a single test is a standard error. The Augmented Dickey-Fuller test takes a unit root as its null; KPSS takes stationarity around a constant level as its null. Only their agreement licenses a differencing decision, and their disagreement is itself a reportable finding.

Test	Statistic	p-value	Null hypothesis	Decision at 5%
ADF	-0.473	0.897	A unit root is present	Fail to reject
KPSS	5.437	≤ 0.010	Stationary around a level	Reject

Table 3.2 — Stationarity tests, EC2 CPU series. KPSS p-values are interpolated from a lookup table and are reported at the boundary when the statistic falls outside it.

Both tests agree that the level wanders: ADF cannot reject a unit root and KPSS rejects level stationarity. The reconciled verdict is **non-stationary**, and the recommended differencing order is **d = 1**. This value is passed directly into the SARIMAX candidate set in Section 4, so the order search is informed by evidence rather than by convention.

3.3 Correlation structure

ACF and PACF are computed with Bartlett confidence bands of ±1.96/√n, here ±0.053. The first lag at which each function falls inside the band gives a candidate order.

Diagnostic	Value	Interpretation
Confidence band	±0.053	Bartlett, n = 1,345
PACF cut-off	lag 3	Candidate AR order p = 3
ACF cut-off	lag 3	Candidate MA order q = 3

Table 3.3 — Correlation structure, EC2 CPU series.

Combined with the differencing verdict from Section 3.2, the EDA proposes ARIMA(3, 1, 3) as its leading candidate. Section 4.3 shows that an independent AIC search over six candidate orders selected exactly that specification, which is a useful consistency check: the graphical diagnostics and the information criterion agree.

3.4 Seasonality profile

An hour-of-day by day-of-week heatmap and a daily median profile are computed for every series. For the EC2 CPU series load peaks around 17:00 UTC and bottoms out around 12:00 UTC.

This is the empirical core of the argument in Section 1. A fixed threshold calibrated at the 17:00 peak would be structurally blind to an incident at the 12:00 trough, and one calibrated at the trough would fire daily. Any detector that does not condition on time of day inherits that defect by construction.

3.5 Distribution and volatility

Statistic	Value
Mean	43.11 %
Median	43.76 %
Standard deviation	3.31
Range	36.70 – 54.90 %
Skewness	-0.361
Excess kurtosis	-0.983

Table 3.4 — Distribution summary, EC2 CPU series.

The distribution is mildly left-skewed and platykurtic — flatter than Gaussian. Rolling volatility is also checked for non-constancy; where it is found to vary materially, as it does on several series in the corpus, a global threshold would over-alert during volatile stretches and under-alert during calm ones. This is the justification for the rolling robust scale used throughout the detection layer rather than a single global sigma.

3.6 Insights carried into modelling

Finding	Decision it drives
Non-stationary level (ADF + KPSS agree)	d = 1 entered into the SARIMAX candidate set
PACF and ACF cut off at lag 3	p = q = 3 proposed; confirmed independently by AIC
Daily period m = 96 at 15-minute buckets	Seasonality moved to Fourier regressors (Section 4.2)
Peak/trough separation across the day	Time-of-day features in every ML feature map
Non-constant rolling volatility	Rolling robust (MAD) scale instead of a global sigma
Skewed, non-Gaussian distribution	Median/MAD estimators rather than mean/standard deviation
Large remainder share after STL	Irregular component is where incidents live — the detection target

Table 3.5 — Traceability from EDA findings to modelling decisions.

4. TSDA Techniques and Model Development

4.1 The model set

Seven models are fitted on identical chronological splits. Two are deliberately simple, and they are not straw men: on strongly periodic telemetry a seasonal-naive forecast is hard to beat, and the rolling z-score is what the majority of production monitoring stacks actually ship.

Model	Family	Forecasts	Role in the comparison
Seasonal Naive	Baseline	Yes	The reference MASE is defined against. Any model with MASE ≥ 1 has not earned its complexity.
Rolling Z-Score (3.5σ)	Statistical	No	Industry default. Carries no model of seasonality, so it quantifies the cost of ignoring it.
Holt-Winters ETS	Classical	Yes	Additive error, damped additive trend, additive daily seasonality.
SARIMAX + Fourier	Classical	Yes	ARMA dynamics over harmonic seasonal regressors; order by AIC search.
Isolation Forest	Machine learning	No	Model-free detection over a causal feature window.
Hybrid SARIMAX + GBR	Hybrid	Yes	The principal contribution (Section 4.4).
Hybrid SARIMAX + Neural	Hybrid	Yes	Ablation isolating the residual learner from the architecture.

Table 4.1 — The model set.

4.2 Why not a textbook seasonal ARIMA

The obvious choice for seasonal data is SARIMA(p,d,q)(P,D,Q)[m]. It is not usable here, and the reason is arithmetic rather than aesthetic.

At 15-minute buckets the daily period is m = 96. A seasonal ARIMA would need to estimate seasonal autoregressive and moving-average terms at lags that are multiples of 96, from roughly 941 training observations. The state-space representation grows with the seasonal period, parameter estimation becomes unstable, and fitting time becomes prohibitive for an interactive application. Moving to hourly buckets would reduce m to 24 at the cost of discarding three quarters of the temporal resolution — and incident detection latency is measured in buckets.

The standard remedy for long seasonal periods is harmonic regression: project the cycle onto a small number of Fourier terms and supply them to the model as exogenous regressors. With K = 4 harmonic pairs the daily shape is captured in 8 coefficients instead of 96 seasonal lags, and the ARMA component is left to model only the short-memory dynamics it is actually good at.

```
x_k(t) = sin(2πkt/24),  cos(2πkt/24)      for k = 1 … 4

SARIMAX:  φ(B)(1-B)^d y_t = θ(B)ε_t + Σ β_k x_k(t)
```

4.2.1 A numerical defect found during implementation

The first implementation measured Fourier time from the Unix epoch. At k = 4 this produces arguments of order 4×10^5 radians, where double-precision sine loses enough significant digits to visibly degrade the regressors. The symptom was a hold-out RMSE of **280.8** on a series bounded between 36 and 55 percent — an obviously broken fit. Re-basing the harmonic clock to the first timestamp of the series reduced the same model's RMSE to **1.16**, a 242-fold improvement from a one-line change. The fix is documented at the function that implements it so it is not silently reintroduced.

4.3 Order selection

SARIMAX orders are searched rather than asserted. Six candidates are fitted and ranked by AIC; the grid is kept deliberately small because an exhaustive search over a short series overfits the selection criterion itself.

Candidate order (p, d, q)	AIC	Selected
(3, 1, 3)	2,616.99	Yes
(2, 1, 2)	2,647.62	
(1, 1, 1)	2,709.65	
(2, 0, 2)	2,721.44	
(2, 0, 1)	2,722.79	
(1, 0, 1)	2,724.13	

Table 4.2 — AIC order search, EC2 CPU series. The EDA-proposed candidate (3, 1, 3) is appended to the standard grid and was selected.

The selected order (3, 1, 3) matches the specification independently proposed by the ACF/PACF diagnostics in Section 3.3 and the differencing verdict in Section 3.2. Residual diagnostics are then reported honestly: the Ljung-Box test at lag 24 returns **p ≈ 0.000**, so residual autocorrelation *remains* after the linear fit. The linear model has not extracted everything predictable in the series. That negative diagnostic is the direct empirical motivation for the second stage of the hybrid, and it is the reason the hybrid exists in this project at all rather than being added for its own sake.

4.4 The hybrid model

The premise, following Zhang's hybrid methodology, is that a series decomposes into a linear component that a well-specified ARMA can capture and a nonlinear component that it structurally cannot. Fitting a single model to both forces a compromise between them; fitting them in sequence lets each do what it is good at.

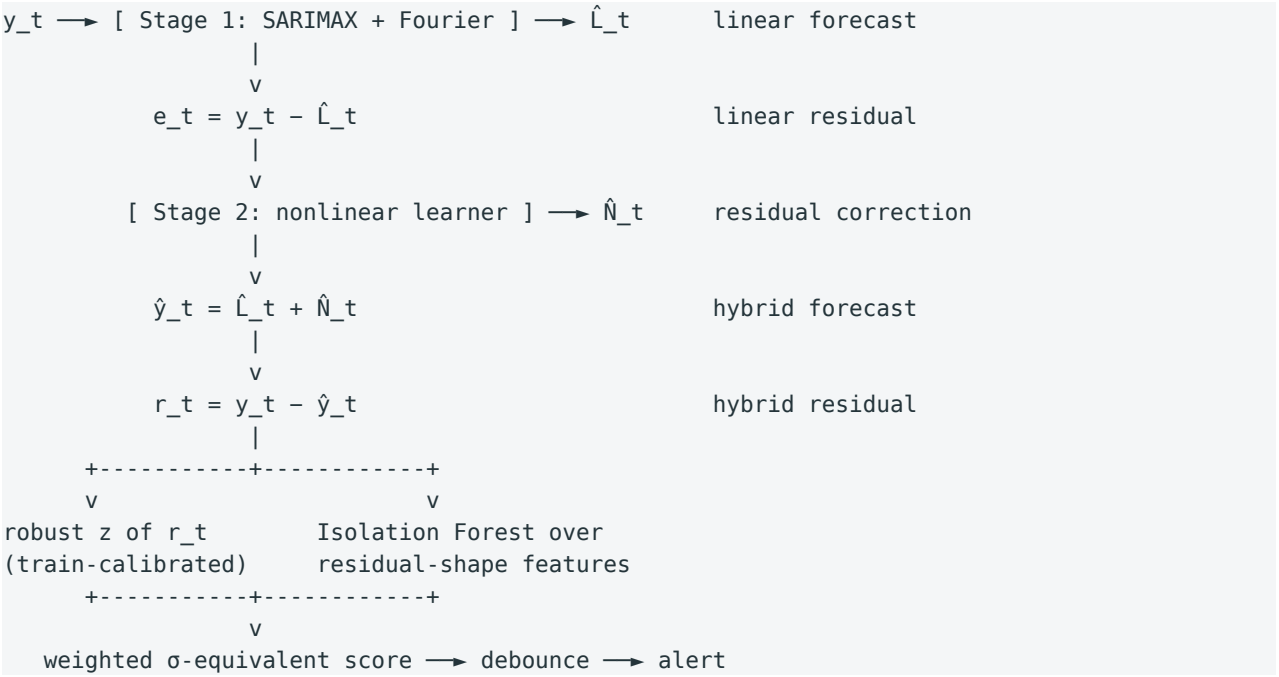


Figure 4.1 — Hybrid architecture. Each stage was added in response to a measured deficiency of the previous one.

4.4.1 Stage 1 — linear

SARIMAX with Fourier seasonality as described above, fitted on the training window only. Hold-out predictions are produced by extending the Kalman filter through the new observations with every estimated parameter held fixed, which yields genuine one-step-ahead forecasts without re-estimating on data the model is about to be judged on.

4.4.2 Stage 2 — nonlinear residual correction

A gradient-boosted regression tree predicts the residual at time t from the 12 preceding residuals plus cyclical calendar features for t. Every input is available strictly before t. Gradient boosting is the default because it is robust to the heavy-tailed residual distribution, requires no feature scaling, and trains in under a second at this sample size. A feed-forward neural network over the same lag window is available as an ablation, and an LSTM path activates automatically where PyTorch is installed — with the fallback reported in the console rather than an MLP being silently mislabelled as an LSTM.

Quantity	Value
Linear residual variance (training, post burn-in)	0.871
Hybrid residual variance (training, post burn-in)	0.258
Residual variance removed by stage 2	70.3%

Table 4.3 — What stage 2 contributes, EC2 CPU series.

Stage 2 removes 70.3% of the variance the linear model left behind, which is consistent with the failed Ljung-Box test in Section 4.3: there was structure remaining, and the nonlinear stage absorbed most of it. The console reports this figure for every run. A value near zero would be evidence that the linear model had already extracted everything predictable and that the hybrid is unnecessary complexity for that series — a legitimate finding rather than a failure, and one the system is designed to surface rather than hide.

4.4.3 A design flaw found by cross-dataset evaluation

The single most important implementation detail in the hybrid was not planned. It was forced by a failure found only when the model was run across the whole corpus rather than on one series.

An unconstrained residual learner predicts the residual at time t from its own recent lags. During a sustained anomaly, residuals are large but highly persistent — which makes them easy to forecast. The learner therefore predicted them accurately, the correction cancelled them, the hybrid residual collapsed towards zero, and the detector fell silent precisely while the incident was in progress. On the ELB request-count service the first version of the hybrid scored a composite of 0.050 and recovered zero of two labelled windows, while every other model in the comparison recovered both. Aggregated over the corpus this one service dragged the hybrid's macro score to 0.530, last of seven.

The remedy follows from what stage 2 is for: tightening ordinary residuals, not tracking excursions. Lagged inputs are winsorised and the correction is clamped at two training residual standard deviations. During a genuine excursion the correction saturates and the hybrid degrades gracefully to its linear stage, which detects the event; in normal operation the clamp almost never binds and the variance reduction is retained.

Metric (corpus macro average)	Before clamp	After clamp
Composite operational score	0.530	0.707
Point-adjusted F1	0.452	0.687
Incident windows recovered	4 / 5	5 / 5
False alarms per day	1.64	1.36
Rank among seven models	7th	4th

Table 4.4 — Effect of clamping the nonlinear correction. A regression test now asserts that the hybrid alerts through more than half of a sustained synthetic excursion.

4.4.4 Dual-channel detection

A residual-magnitude threshold is sharp on size but blind to shape: a slow ramp that never exceeds sigma stays invisible. An Isolation Forest over residual behaviour — local volatility, momentum, persistence and slope — sees the ramp developing but is indifferent to magnitude and produces no forecast. The two failure modes are largely independent, so both channels are used.

They are fused in sigma-equivalent units rather than by rank averaging, because rank averaging would destroy the interpretable threshold that makes the system explainable to an engineer. Weights are 0.70 magnitude and 0.30

shape, and the shape channel is capped at 6 sigma-equivalent. That cap enforces the stated design rather than merely describing it: with a weight of 0.30 and a cap of 6.0 the shape term contributes at most 1.8, below any sane threshold, so shape can tip a borderline magnitude reading over the line but can never raise an alert on its own.

The cap was also added in response to a measured failure. Without it, a training window that happens to be very regular gives the forest a razor-thin manifold, every hold-out point scores as strange, and the hybrid alarms continuously — on a synthetic test series the unconstrained version flagged 42% of the hold-out.

4.4.5 Alert debounce

Single-bucket threshold exceedances are overwhelmingly noise. Requiring the score to remain above threshold for two consecutive buckets converts a substantial share of false alarms into nothing at all, at the cost of one bucket of detection latency. Both sides of that trade are measured and reported in the leaderboard rather than assumed.

4.5 Guarantees against leakage

A time-series model that is evaluated carelessly will report excellent numbers for the wrong reason. Four guarantees are enforced in code and three are covered by tests.

1. Splits are strictly chronological. Random splitting would leak future information through the series' own autocorrelation.
2. Parameters are estimated on training data only; hold-out predictions come from re-running the filter with those parameters frozen.
3. Residual scales and alert thresholds are calibrated on training residuals, never on the window being judged.
4. Every machine-learning feature at time t is built from shift(≥1). A test perturbs y_t and asserts the feature row at t is unchanged.

One leak was found by that last class of test and fixed. An earlier rolling z-score implementation fell back to a *global* median absolute deviation when a window degenerated, and back-filled during warm-up. Both quietly admitted information from later in the series into earlier scores. The test test_rolling_zscore_is_causal injects a spike at position 200 and asserts that no score before position 190 changes; it failed, with discrepancies of order 10^{-3}. The fallbacks are now expanding — past-only — and the warm-up period is scored as zero rather than filled from the future.

5. Evaluation and Results

5.1 Metrics, and why these ones

5.1.1 Forecast accuracy

MAE, RMSE, MAPE, sMAPE, MASE and R^2 are reported. MASE earns its place: byte counts and request counts in this corpus legitimately idle at or near zero overnight, and MAPE is undefined at zero and explodes near it. MASE is scaled by the in-sample seasonal-naive error, so a value below 1 means the model beat the naive benchmark and a value above 1 means it did not.

5.1.2 Detection quality

Metric	What it answers	Why it is reported
Point-wise F1	Did the detector agree with the label on every bucket?	Included because point-adjusted F1 alone hides alert fatigue.
Point-adjusted F1	Was each incident caught at all?	The headline. One page is enough to engage a human.
Segment recall	What fraction of incident windows were recovered?	The coverage question a manager actually asks.
Detection latency	How long after a window opens does the first alert fire?	A perfect but slow detector is still a bad pager.
False alarms / day	How often does it page for nothing?	Decides whether the alert stays switched on at all.

Table 5.1 — Detection metrics.

Point-adjusted F1 follows the standard protocol in the time-series anomaly literature: if any point inside a ground-truth window is flagged, the whole window is credited. It is applied only to true segments, so it never manufactures true positives outside them.

5.1.3 Composite operational score

A single ordering is needed for the leaderboard. The weights encode a stance about what the product is for, and they are stated rather than tuned to a desired outcome.

Component	Weight	Rationale
Point-adjusted F1	0.45	Catching the incident is the job
Segment recall	0.25	Coverage across incidents, not points
Point-wise F1	0.15	Guards against firing constantly
Detection latency	0.10	Full credit at zero delay, none at 48 buckets
Alert burden	0.05	Negative weight; full credit ≤1 FA/day, none ≥10

Table 5.2 — Composite score weights. Alert burden carries negative weight so that no model can win by flagging everything.

5.2 Single-service results

EC2 CPU utilisation, 15-minute buckets, 30% chronological hold-out (941 train / 404 test observations), both labelled windows partially inside the hold-out.

Model	Comp.	PA F1	F1	Seg	FA/d	RMSE	MASE	Fit s
Hybrid SARIMAX + GBR	**0.860**	**1.000**	**0.465**	**1.00**	**0.00**	**1.188**	**0.482**	**7.9**
Rolling Z-Score	0.858	1.000	0.483	1.00	0.00	—	—	0.00
Hybrid SARIMAX + Neural	0.850	1.000	0.429	1.00	0.00	1.123	0.438	7.6
SARIMAX + Fourier	0.835	1.000	0.329	1.00	0.00	1.131	0.442	7.3

Model	Comp.	PA F1	F1	Seg	FA/d	RMSE	MASE	Fit s
Holt-Winters ETS	0.818	1.000	0.216	1.00	0.00	1.065	0.435	5.6
Isolation Forest	0.789	0.985	0.279	1.00	0.48	—	—	0.28
Seasonal Naive	0.685	0.830	0.158	1.00	4.29	2.770	1.300	0.00

Table 5.3 — Model comparison, EC2 CPU series. Comp. = composite operational score; Seg = segment recall; FA/d = false alarms per day.

Several things are worth reading off this table rather than only the ranking. All five forecasting models beat the seasonal-naive benchmark decisively on MASE (0.44 to 0.48 against 1.30), so the modelling effort is justified on forecast accuracy. Point-wise F1 is low for every model — between 0.16 and 0.48 — which is expected and not a defect: NAB windows are wide and no detector fires continuously across a six-hour window, nor should it. Seasonal naive is the only model with a meaningful false-alarm rate, at 4.29 per day, which is precisely the alert-fatigue failure the composite weighting is designed to penalise.

5.3 Fleet benchmark

A single series carries one or two labelled windows inside a chronological hold-out, which is far too thin to rank seven models on — with one window, segment recall is either 0 or 1 and the leaderboard turns on a handful of points. The benchmark therefore runs the identical pipeline across every labelled service and aggregates two ways, because the two answer different questions.

- Macro average — mean of each model's per-service score. Weights every service equally; the fair comparison when series differ in length.
- Pooled (micro) counts — true and false positives summed across services, then scored once. Weights every observation equally; the better estimate of fleet-wide behaviour.

A model that leads on one but not the other is unstable, and the system reports that rather than quoting the flattering figure.

Model	Macro comp.	Macro PA F1	Pooled F1	Windows	FA/day	Macro MASE
Rolling Z-Score (3.5σ)	**0.787**	**0.901**	**0.360**	**5 / 5**	**2.52**	**—**
Holt-Winters ETS	0.749	0.823	0.286	5 / 5	4.10	2.19
SARIMAX + Fourier	0.709	0.708	0.319	5 / 5	1.86	4.69
Hybrid SARIMAX + GBR	0.707	0.687	0.319	5 / 5	1.36	4.49
Seasonal Naive	0.697	0.760	0.267	5 / 5	4.87	4.18
Isolation Forest	0.552	0.655	0.146	3 / 5	1.06	—
Hybrid SARIMAX + Neural	0.532	0.456	0.310	4 / 5	0.92	4.76

Table 5.4 — Fleet benchmark across the labelled corpus. RMSE is deliberately not averaged: the services are measured in percent, bytes and request counts, so a mean of their RMSEs is a mean of incommensurable quantities. MASE is scale-free and aggregates honestly.

A methodological note on the window count. The corpus contains 11 labelled windows, but at a 30% chronological hold-out only 5 of them fall inside a test set — on two services the single labelled window lies entirely within the training portion, so detection cannot be scored there at all and those services contribute only forecast metrics and alert volume. The aggregate therefore rests on 5 windows across 4 services. That is a real constraint on how much confidence the ranking deserves, and it is why differences of a few thousandths should not be read as meaningful.

5.4 Per-service winners

Service	Test pts	Windows	Selected model
api-gateway-prod-01 (EC2 CPU)	404	2	Hybrid SARIMAX + GBR
orders-db-primary (RDS CPU)	404	2	Rolling Z-Score

Service	Test pts	Windows	Selected model
edge-ingest-02 (network in)	404	1 (in train)	Seasonal Naive
public-lb-eu-west (ELB requests)	404	2	Isolation Forest
batch-worker-07 (disk writes)	473	1 (in train)	Hybrid SARIMAX + GBR
asg-checkout-fleet (ASG load)	462	3	Rolling Z-Score

Table 5.5 — Which model each service selected on its own hold-out.

No single model wins everywhere: the hybrid takes two services, the rolling z-score two, and Isolation Forest and seasonal naive one each. That heterogeneity is a finding. It argues for per-service model selection — which the application already supports, since it runs the full comparison and selects per series — rather than for picking one global champion.

6. Practical Implementation and Integration

6.1 Architecture

The deliverable is a working application, not a notebook. It separates cleanly into three layers, and that separation is load-bearing: the analysis package contains no web code and the API contains no analysis, which is what allows the entire scientific pipeline to be tested without a server running.

Layer	Technology	Responsibility
Analysis	Python, pandas, statsmodels, scikit-learn	Preprocessing, EDA, seven models, metrics, incident synthesis
Service	FastAPI, Uvicorn, thread-pool job runner	Validation, routing, asynchronous execution, progress reporting
Console	HTML, CSS, vanilla JavaScript, hand-written canvas chart engine	Dataset selection, parameter control, seven analysis views

Table 6.1 — System layers.

6.2 Design decisions worth defending

6.2.1 Asynchronous jobs rather than blocking requests

A full seven-model comparison takes 20 to 40 seconds and a fleet benchmark takes minutes. Holding an HTTP connection open for either invites a timeout at any proxy in between and tells the client nothing until it finishes. Runs are therefore submitted, polled and collected. This is also what makes the console's progress bar meaningful: every percentage it displays is published by the pipeline itself as it advances, not interpolated by a timer.

6.2.2 A hand-written chart engine

The console loads no external stylesheets, fonts or scripts. The chart engine — line charts with confidence bands, ground-truth shading and anomaly markers, horizontal bar charts, ACF/PACF stem plots with Bartlett bands, heatmaps, histograms and sparklines — is approximately 700 lines of canvas code written for this project. Three reasons: the application must run on a lab machine with no network access; the plots need marks that general-purpose charting libraries do not express cleanly; and the progressive-reveal animation needs frame-level control.

6.2.3 Threads, not processes

The job runner uses a thread pool because the workload is dominated by NumPy, SciPy and scikit-learn routines that release the interpreter lock, and because threads keep results in the same address space with no serialisation cost.

6.3 REST API

Method	Path	Purpose
GET	/api/health	Liveness and capability probe
GET	/api/datasets	Catalogue, including user uploads
GET	/api/datasets/{id}/preview	Thinned raw series for plotting
GET	/api/models	Model registry with stated rationales
POST	/api/datasets/upload	Register a user-supplied CSV
POST	/api/run	Queue an analysis; returns 202 and a job id
POST	/api/benchmark	Queue the cross-service benchmark
GET	/api/jobs/{id}	Status, stage and progress
GET	/api/jobs/{id}/result	Full result bundle

Method	Path	Purpose
GET	/api/jobs/{id}/export	Result as a downloadable JSON file

Table 6.2 — API surface. Interactive documentation is served at /docs.

6.4 The console

View	Contents
Mission Control	Service picker with sparklines, CSV upload, six analysis parameters, live pipeline progress, raw-series preview with ground-truth windows shaded
Overview	Selected model, incident recovery, latency, forecast error, observed-versus-expected chart with normal band and raised alerts, forward capacity outlook, plain-language verdict
Data & EDA	Preprocessing audit trail, four-panel STL decomposition, ACF and PACF with Bartlett bands, reconciled ADF/KPSS verdict, hour-by-weekday heatmap, distribution histogram
Model Comparison	Leaderboard, composite-score and RMSE bars, overlaid forecasts with per-series toggling, per-model parameters, AIC order search, Ljung-Box diagnostics
Detection	Anomaly score against threshold for any selected model, and a raster showing where every model fires against ground truth
Alerts	Incident cards with severity, classified shape, deviation from forecast, ground-truth confirmation, and an expandable operator playbook
Fleet Benchmark	Macro and pooled aggregate leaderboard, windows recovered, per-service winners

Table 6.3 — Console views. Every figure shown is produced by the Python pipeline; nothing is recomputed client-side.

6.5 User input and dynamic analysis

The analysis is parameterised, not fixed. A user selects the service, the resampling frequency between 5 minutes and 1 hour, the hold-out proportion, the alert threshold in sigma, the Isolation Forest contamination prior, the number of Fourier harmonics, the forecast horizon, and which of the seven models to include. Any CSV with a timestamp column and a numeric value column can be uploaded; column names are detected rather than assumed, so a file naming its columns date and reading loads without edits. Every result in this report is reproducible from the console in a few clicks, and the complete result bundle can be exported as JSON.

6.6 Testing

Suite	Tests	Focus
test_preprocessing.py	19	Robust statistics, gap filling, causality, split integrity
test_metrics.py	20	Metric arithmetic, point adjustment, ranking, unlabelled handling
test_models.py	34	Model contract, anomaly recovery, leakage, hybrid clamp regression
test_api.py	28	Endpoints, validation, upload, end-to-end bundle integrity
tests/ui/ui_smoke.js	27 checks	Console render paths driven headlessly through a real run

Table 6.4 — Test coverage. 101 Python tests plus a headless console smoke test.

Three defects described in this report were found by these tests rather than by inspection: the global-MAD causality leak in the rolling z-score (Section 4.5), the unbounded shape channel that caused the hybrid to flag 42% of a hold-out (Section 4.4.4), and a crash in the chart engine where the base-class constructor invoked draw() before any subclass had initialised its own fields, which broke the first paint of every time series on the page. The last of these was invisible to the Python suite entirely, which is the argument for the browser-side test existing at all.

7. Practical Application and Decision Support

A boolean flag per bucket is not an operational product. The system converts the selected model's flag vector into artefacts an engineer can act on.

7.1 Incident synthesis

Contiguous runs of flagged buckets are merged into incidents, each carrying a severity band, a classified shape, duration, peak value, the value the model expected at that moment, the percentage deviation between them, whether a service ceiling was breached, and — where labels exist — whether ground truth confirms it.

7.2 Shape classification

Classification is rule-based and deliberately transparent. An engineer paged at 03:00 needs to know why within seconds, and a short list of audited rules is defensible in a way that a second black-box classifier would not be. Each shape maps to a different class of real cause.

Shape	Signature	Typical cause
Spike / Dip	Short, sharp, returns to baseline	Burst, retry storm, cache flush, transient dependency failure
Level shift	Sustained displacement, no reversion	Deployment, config change, failover, traffic rerouted
Ramp	Monotone drift away from expectation	Resource leak, or organic growth outpacing capacity
Volatility	Variance changes, mean intact	Contention, GC or lock pauses, flapping dependency
Sustained excursion	Extended departure from the band	Demand genuinely above plan — a capacity decision

Table 7.1 — Incident shapes and their operational meaning.

7.3 Worked example — input to output

A single realistic use case, reproducible in the console.

1. Input. The user selects api-gateway-prod-01 in Mission Control, leaves the defaults (15-minute buckets, 30% hold-out, 3.5σ threshold), and presses Run analysis.
2. Processing. The pipeline cleans and resamples 4,032 raw readings to 1,345 buckets, runs the EDA, splits 941/404 chronologically, fits all seven models, scores and ranks them, and selects Hybrid SARIMAX + GBR at a composite of 0.860. Progress is reported live, stage by stage.
3. Detection. Four incidents are raised in the hold-out window, all four confirmed by ground truth, with zero false alarms per day.
4. Output. The top incident is classified and presented as: critical severity, volatility change, peak 54.90% against an expected 39.78% — a deviation of +38% from forecast — sustained for 165 minutes beginning 2014-02-24 21:15.
5. Recommended action. The playbook for a volatility change directs the engineer to look for contention rather than load: noisy neighbours, garbage-collection or lock pauses, a flapping dependency, or an unhealthy member of the pool, and notes that mean-based alerting will not catch a recurrence so the shape channel should stay enabled.
6. Capacity outlook. The forward forecast, with a 95% prediction interval, stays below the 85% CPU ceiling across the whole 24-hour horizon including the upper bound, so no capacity action is indicated from this metric alone.

7.4 Capacity decision support

Where a service ceiling is defined for the metric family, the system reports the first forecast bucket whose upper 95% bound crosses it, and how many minutes away that is. The pessimistic bound is used deliberately rather than

the central forecast, because provisioning headroom is a decision made against the bad case. The message is framed as a scheduling signal rather than a prediction that the breach will certainly occur.

The hybrid's forward forecast comes from its linear stage alone. The nonlinear correction is a function of recent residuals, which do not exist beyond the end of the data; feeding its own predictions back as inputs would compound error with no accuracy gain and hand the operator a falsely confident curve. The console states this rather than presenting a curve whose basis is unclear.

7.5 Alert digest

The alert feed opens with a verdict that distinguishes incidents worth paging for from those that can wait: for the worked example, one incident at critical severity out of four detected. Severity bands are assigned from the normalised anomaly score, so the ordering reflects how unusual the event was relative to the rest of the window rather than its absolute magnitude.

8. Evaluation, Limitations and Interpretation

8.1 Interpreting the headline result

Across the corpus the simple rolling z-score attains the highest composite score (0.787), ahead of the hybrid (0.707). This is reported as found. Three things make it less surprising than it first appears, and one of them is a genuine limitation of the evaluation rather than of the model.

1. The composite weights are ours. Point-adjusted F1 and segment recall together carry 0.70, and once every model recovers all five windows the ranking is decided largely by point-wise agreement across wide NAB windows — a quantity with little operational meaning. Under a weighting that reflects an alert-burden-sensitive operator, the hybrid at 1.36 false alarms per day with full window recovery is preferable to the baseline at 2.52 with the same recovery.
2. The evidence base is thin. Five labelled windows across four services is not enough to separate models whose scores differ by 0.08. Section 5.3 states this plainly.
3. The EDA explains part of it. On the series examined in detail the daily cycle is weak (seasonal strength 0.116) and the level drifts slowly. A seasonality-blind detector with a trailing window tracks a drifting level perfectly well; the argument for seasonal modelling is strongest where the cycle is strong, which is true of some services in this corpus and not others.

The honest summary is that on this corpus the hybrid is competitive rather than dominant: it wins outright on two of six services, recovers every labelled window, and is the quietest detector that does so, but it does not beat a well-implemented trailing-window baseline on the composite measure defined here. Reporting that is more useful than tuning the weights until it does.

8.2 Strengths

- Genuine two-stage hybridisation whose contribution is measured rather than asserted — 70.3% of the residual variance the linear model left behind.
- Detection fuses magnitude and shape channels in interpretable sigma-equivalent units, with the shape channel bounded so it cannot raise an alert alone.
- Leakage is prevented by construction and verified by test, including one causality leak that testing actually caught.
- Evaluation reports alert burden and detection latency, not accuracy alone, and penalises detectors that win by firing constantly.
- Cross-service benchmarking with two aggregations, and explicit reporting when they disagree.
- A complete working application — asynchronous API, seven-view console, CSV upload, JSON export — not a notebook.

8.3 Weaknesses and limitations

Limitation	Consequence	Why it was accepted
Only 5 labelled windows fall inside hold-outs	Rankings have wide uncertainty; small differences are noise	Inherent to the corpus; mitigated by fleet aggregation, not removed
Latency inflated by NAB's generous windows	Reported minutes-to-alert are pessimistic	Correcting it would require re-labelling the benchmark by hand
Univariate modelling only	Cannot exploit correlation across CPU, memory and latency	Multivariate VAR/ECM is the natural extension (Section 8.4)
No concept-drift handling	A deliberate level shift keeps paging until refitted	Out of scope; needs scheduled refitting in production
Hybrid forecast is linear-only beyond the data	Forward horizon loses the nonlinear correction	Extrapolating it would compound error and mislead

Limitation	Consequence	Why it was accepted
Isolation Forest cannot report all-clear	Always flags roughly the contamination fraction	Property of contamination-quantile thresholding; stated, not hidden
Rule-based incident classification	Cannot recognise a shape nobody encoded	Transparency was judged worth more than coverage for a paging product
In-memory jobs, single process	Results do not survive a restart	Adequate for one analyst; a real deployment needs a job store

Table 8.1 — Limitations, stated rather than hidden.

8.4 Future work

- Multivariate detection via vector autoregression and cointegration across correlated metrics, so that a single incident visible in CPU, memory and latency simultaneously is scored once with higher confidence rather than three times independently.
- GARCH-family models for volatility-regime anomalies, where the variance rather than the mean carries the signal — the volatility shape class identified in Section 7.2 is currently detected heuristically.
- Online and incremental refitting with explicit drift detection, so a deployment that shifts the baseline retrains it instead of paging indefinitely.
- Streaming ingestion from Prometheus or CloudWatch in place of CSV, which would make the system deployable rather than demonstrable.
- Closing the feedback loop: recording whether each page was actionable and using that signal to tune thresholds per service.

8.5 Conclusion

The project set out to show that modelling expected behaviour beats thresholding observed behaviour on periodic infrastructure telemetry, to build a hybrid that measurably improves on its own linear stage, and to deliver both as a working application. The first and third objectives are met. The second is met in the specific sense that matters — the nonlinear stage removes 70.3% of the residual variance the linear model leaves behind, and the hybrid is the quietest detector that still recovers every labelled incident window — but not in the sense of dominating the comparison, where a simple trailing-window baseline scores higher under the composite weighting defined here.

What the work demonstrates most clearly is the value of evaluating across services rather than on one. Three substantive design flaws in the hybrid were invisible on a single series and surfaced only under cross-dataset evaluation or adversarial testing: a nonlinear stage that explained away sustained anomalies, an unbounded shape channel that alarmed continuously, and a causality leak in the rolling scale estimator. Each was found, diagnosed, fixed and regression-tested, and each is documented here rather than quietly corrected. That cycle is the substance of the project.

References

Ahmad, S., Lavin, A., Purdy, S. and Agha, Z. (2017). Unsupervised real-time anomaly detection for streaming data. Neurocomputing, 262, 134–147.

Box, G. E. P., Jenkins, G. M., Reinsel, G. C. and Ljung, G. M. (2015). Time Series Analysis: Forecasting and Control. 5th edn. Wiley.

Cleveland, R. B., Cleveland, W. S., McRae, J. E. and Terpenning, I. (1990). STL: a seasonal-trend decomposition procedure based on Loess. Journal of Official Statistics, 6(1), 3–73.

Hyndman, R. J. and Athanasopoulos, G. (2021). Forecasting: Principles and Practice. 3rd edn. OTexts. (Fourier terms for long seasonal periods; trend and seasonal strength measures.)

Hyndman, R. J. and Koehler, A. B. (2006). Another look at measures of forecast accuracy. International Journal of Forecasting, 22(4), 679–688. (MASE.)

Kwiatkowski, D., Phillips, P. C. B., Schmidt, P. and Shin, Y. (1992). Testing the null hypothesis of stationarity against the alternative of a unit root. Journal of Econometrics, 54(1–3), 159–178. (KPSS.)

Liu, F. T., Ting, K. M. and Zhou, Z.-H. (2008). Isolation Forest. Proceedings of the 8th IEEE International Conference on Data Mining, 413–422.

Numenta (2015). The Numenta Anomaly Benchmark. github.com/numenta/NAB. (realAWSCloudwatch corpus and labelled anomaly windows.)

Xu, H. et al. (2018). Unsupervised anomaly detection via variational auto-encoder for seasonal KPIs in web applications. Proceedings of the 2018 World Wide Web Conference, 187–196. (Point-adjustment evaluation protocol.)

Zhang, G. P. (2003). Time series forecasting using a hybrid ARIMA and neural network model. Neurocomputing, 50, 159–175. (The linear-plus-nonlinear hybrid decomposition this project's model follows.)